



**Mahatma Education Society's  
Pillai College of Engineering  
(Autonomous) Accredited A+ by NAAC  
Dr. K. M. Vasudevan Pillai Campus,  
Plot No. 10, Sector-16, New Panvel - 410206**

A Report on

## **Forward Error Control (FEC) Simulation**

A MATLAB-Based Error Control Simulation

For

**Digital Communication (TY, Sem V)**

In

**Electronics & Telecommunication Engineering**

Project by:

**ETCA533 – Vedant Mishra**

**ETCA531 – Anuj Meher**

Under the guidance of

**Dr. Suman Wadkar**

Mentor's sign

## TABLE OF CONTENTS

| No. | Section                          | Page  |
|-----|----------------------------------|-------|
| 1   | Abstract                         | 3     |
| 2   | Introduction                     | 4     |
| 3   | Problem Statement and Objectives | 5     |
| 4   | Mathematical Formulation         | 6     |
| 5   | System Overview                  | 7     |
| 6   | Simulation Methodology           | 8-9   |
| 7   | MATLAB Implementation            | 10    |
| 8   | Results and Analysis             | 11-12 |
| 9   | Advantages and Limitations       | 13    |
| 10  | Future Scope                     | 14    |
| 11  | Conclusion                       | 15    |
| 12  | References                       | 16    |

## 1. ABSTRACT

This project simulates a digital communication system utilizing Forward Error Control (FEC) to ensure data integrity across a noisy transmission medium. A Hamming (7,4) error-correcting code was implemented in MATLAB to encode a 4-bit message, transmit it through an Additive White Gaussian Noise (AWGN) channel, and dynamically correct single-bit errors at the receiver. The system's performance was evaluated through stage-by-stage waveform visualization and a Bit Error Rate (BER) versus Signal-to-Noise Ratio (SNR) statistical evaluation.

## 2. INTRODUCTION

In digital communications, physical channels are susceptible to environmental interference, which can flip bits and corrupt data payloads. Forward Error Control (FEC) mitigates this by embedding mathematical redundancies (parity bits) into the transmitted signal, allowing the receiver to detect and correct errors without requiring the sender to retransmit the data. This simulation demonstrates the operational mechanics of a block code system, specifically the Hamming (7,4) algorithm, validating its ability to maintain data integrity under varying noise conditions.

## **3. PROBLEM STATEMENT AND OBJECTIVES**

### **3.1 Problem Statement**

Digital communication systems operating over physical channels inevitably experience bit errors due to Additive White Gaussian Noise (AWGN), attenuation, and environmental interference. Without robust error-control mechanisms, these corrupted bits compromise payload integrity, leading to communication breakdowns or critical data failures at the receiving end.

### **3.2 Objectives**

- Design and mathematically model a Hamming (7,4) Forward Error Control system.
- Simulate a realistic communication channel by injecting AWGN into baseband signals.
- Visualize the encoding, error injection, and decoding stages using discrete digital waveforms.
- Quantify the statistical performance and efficiency of the system by generating a comprehensive Bit Error Rate (BER) versus Signal-to-Noise Ratio (SNR) curve.

## 4. MATHEMATICAL FORMULATION

The Hamming (7,4) code processes 4 data bits ( $D_1, D_2, D_3, D_4$ ) and generates 3 parity bits ( $P_1, P_2, P_3$ ) to create a 7-bit codeword. The system utilizes even parity, calculated via modulo-2 arithmetic (XOR logic).

**Encoding (Parity Equations):**

$$P_1 = D_1 \oplus D_2 \oplus D_4$$

$$P_2 = D_1 \oplus D_3 \oplus D_4$$

$$P_3 = D_2 \oplus D_3 \oplus D_4$$

**Decoding (Syndrome Equations):** At the receiver, the integrity of the incoming sequence is verified by recalculating the parity against the received bits to generate a 3-bit Syndrome ( $S_1, S_2, S_3$ ).

$$S_1 = D_1 \oplus D_2 \oplus D_4 \oplus P_1$$

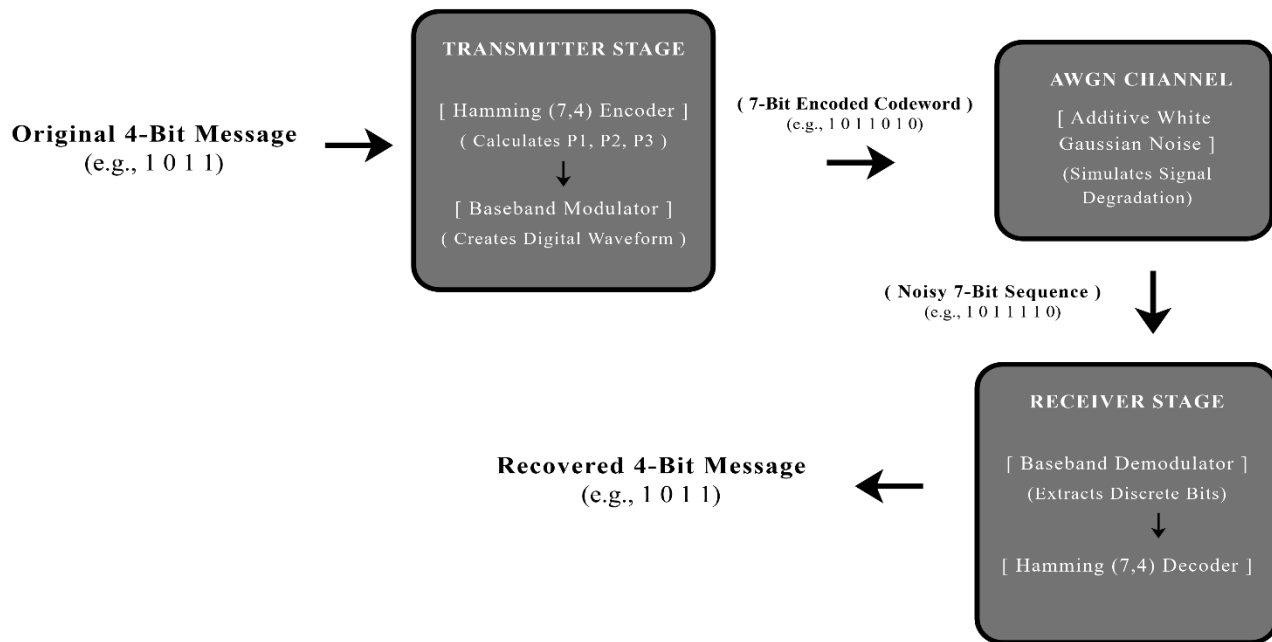
$$S_2 = D_1 \oplus D_3 \oplus D_4 \oplus P_2$$

$$S_3 = D_2 \oplus D_3 \oplus D_4 \oplus P_3$$

If the syndrome is [0,0,0], no error is present. Any non-zero binary value corresponds to the exact bit index that was inverted by channel noise, allowing for immediate mathematical correction.

## 5. SYSTEM OVERVIEW

The architecture is divided into three distinct operational blocks:



**Figure 1: Overall block diagram of the Forward Error Control system**

### 5.1 Transmitter Stage

The data source generates a 4-bit binary sequence. The Hamming Encoder applies the parity equations to append the 3 redundant bits, creating the 7-bit codeword. The sequence is then converted into a physical digital waveform (baseband signal) mapped to binary states for transmission.

### 5.2 AWGN Channel

The communication medium introduces random signal degradation. To simulate real-world transmission interference, Additive White Gaussian Noise (AWGN) is applied to the baseband waveform.

### 5.3 Receiver Stage

The receiver converts the noisy physical waveform back into a discrete binary sequence. The Hamming Decoder calculates the syndrome bits. By mapping the syndrome to the error vector, it mathematically isolates the corrupted bit, inverts it back to its original state, and strips the parity bits to recover the pristine 4-bit payload.

## 6. SIMULATION METHODOLOGY

The simulation workflow follows a sequential data pipeline through the transmitter, channel, and receiver stages to process the data payload.

### 6.1 Input Generation & Encoding:

The system initializes by prompting the user to dynamically input a custom 4-bit binary message array via the MATLAB command window (as shown in image\_8bc049.png). A robust validation loop is implemented to ensure the input strictly conforms to a 4-bit length and contains only valid binary digits. Using modulo-2 arithmetic to simulate hardware XOR gates, three even-parity bits are calculated and appended to the validated data. This creates the 7-bit Hamming codeword, which is then mapped as a digital baseband step-waveform for transmission.

```
%% 1. TRANSMITTER STAGE
% Prompt the user for custom data
D = input('Enter a 4-bit binary message as an array (e.g., [1, 0, 1, 1]): ');

% Validation check to ensure strict 4-bit binary format
while length(D) ~= 4 || ~all(ismember(D, [0, 1]))
    disp('Error: Invalid format. You must enter exactly four binary digits. ');
    D = input('Please enter a valid 4-bit array (e.g., [1, 0, 1, 1]): ');
end

% Calculate Parity Bits
P1 = mod(D(1) + D(2) + D(4), 2);
P2 = mod(D(1) + D(3) + D(4), 2);
P3 = mod(D(2) + D(3) + D(4), 2);

codeword = [D, P1, P2, P3]; % 7-bit codeword

% Initialize the main figure and plot the first graph (Top)
```

Figure 2: MATLAB Editor showing the lines of code for "TRANSMITTER STAGE"

### 6.2 Channel Interference & Noise Injection (AWGN)

The transmitted codeword enters the simulated physical channel where environmental degradation occurs. To test the error-correction logic dynamically, the simulation utilizes a randomization function to arbitrarily select a bit index between 1 and 7 (detailed in image\_8bc085.png). A logical NOT operator (~) mathematically forces a bit-flip at this randomly generated position, representing an unpredictable interference spike corrupting the payload. The waveform visualization is programmed to dynamically update its title to explicitly identify the exact location of the randomized error.

```

%% 2. CHANNEL STAGE
received_seq = codeword;

% Generate a random error position between 1 and 7
error_pos = randi([1, 7]);

% Simulate interference by flipping the randomly chosen bit
received_seq(error_pos) = ~received_seq(error_pos);

% Plot the second graph (Middle)
subplot(3,1,2);
stairs([received_seq, received_seq(end)], 'r', 'LineWidth', 2);

% Dynamically update the title to display the exact randomized error position
title(['Channel: Noisy Received Signal (Random Error at bit ', num2str(error_pos), ')']);
xlabel('Bit Position'); ylabel('Binary State');
ylim([-0.5 1.5]); grid on;

```

Figure 3: MATLAB Editor showing the lines of code for "CHANNEL STAGE"

### 6.3 Decoding & Error Correction (Receiver)

The receiver ingests the corrupted baseband signal and re-applies the parity equations against the received sequence to calculate a 3-bit syndrome. The system uses an (*if-elseif*) logic block to map the non-zero syndrome directly to the error vector. Once the algorithm identifies the corrupted index, it inverts the bit back to its original state, recovering the exact original 4-bit message.

```

%% 3. RECEIVER STAGE
% Calculate Syndrome
S1 = mod(received_seq(1) + received_seq(2) + received_seq(4) + received_seq(5), 2);
S2 = mod(received_seq(1) + received_seq(3) + received_seq(4) + received_seq(6), 2);
S3 = mod(received_seq(2) + received_seq(3) + received_seq(4) + received_seq(7), 2);
syndrome = [S1, S2, S3];

% Locate the error
error_index = 0;
if isequal(syndrome, [1, 1, 0])
    error_index = 1;
elseif isequal(syndrome, [1, 0, 1])
    error_index = 2;
elseif isequal(syndrome, [0, 1, 1])
    error_index = 3;
elseif isequal(syndrome, [1, 1, 1])
    error_index = 4;
elseif isequal(syndrome, [1, 0, 0])
    error_index = 5;
elseif isequal(syndrome, [0, 1, 0])
    error_index = 6;
elseif isequal(syndrome, [0, 0, 1])
    error_index = 7;
end

```

Figure 4: MATLAB Editor showing the lines of code for "RECEIVER STAGE"

## **7. MATLAB IMPLEMENTATION**

The software architecture leverages core MATLAB functions for mathematical logic, signal processing, and performance visualization, avoiding reliance on pre-built communications toolboxes to demonstrate fundamental algorithm mechanics.

### **7.1 Logical Parity & Syndrome Generation**

Modulo-2 arithmetic, executed via the `mod(..., 2)` function, is utilized exclusively for encoding and decoding. This mathematically replicates the behaviour of hardware XOR gates, determining even parity for the transmitted codeword and generating the receiver's error-detection syndrome.

### **7.2 Channel Noise Injection**

The built-in `awgn()` function models real-world physical layer degradation. It calculates the necessary noise variance and injects it into the baseband signal based on the dynamically looping SNR parameter, converting the binary sequence into a continuous noisy signal before receiver digitization.

### **7.3 Data Visualization**

The `stairs()` function generates standard digital step-waveforms for precise bit-level tracking across the transmitter, channel, and receiver stages. For the statistical analysis, the `semilogy()` function maps the exponential drop in the Bit Error Rate on a logarithmic y-axis scale against the linear SNR x-axis.

# 8. RESULT AND ANALYSIS

The system utilizes a dual-approach simulation strategy to evaluate both the logical execution and the statistical performance of the Hamming (7,4) code.

## 8.1 Deterministic Logic Validation

A localized test is conducted by transmitting a single 4-bit message across the system. Environmental interference is simulated by forcing a deliberate bit-flip at a known index within the channel. This methodology provides discrete, stage-by-stage waveform tracking to visually prove the syndrome calculation and bit-inversion logic.

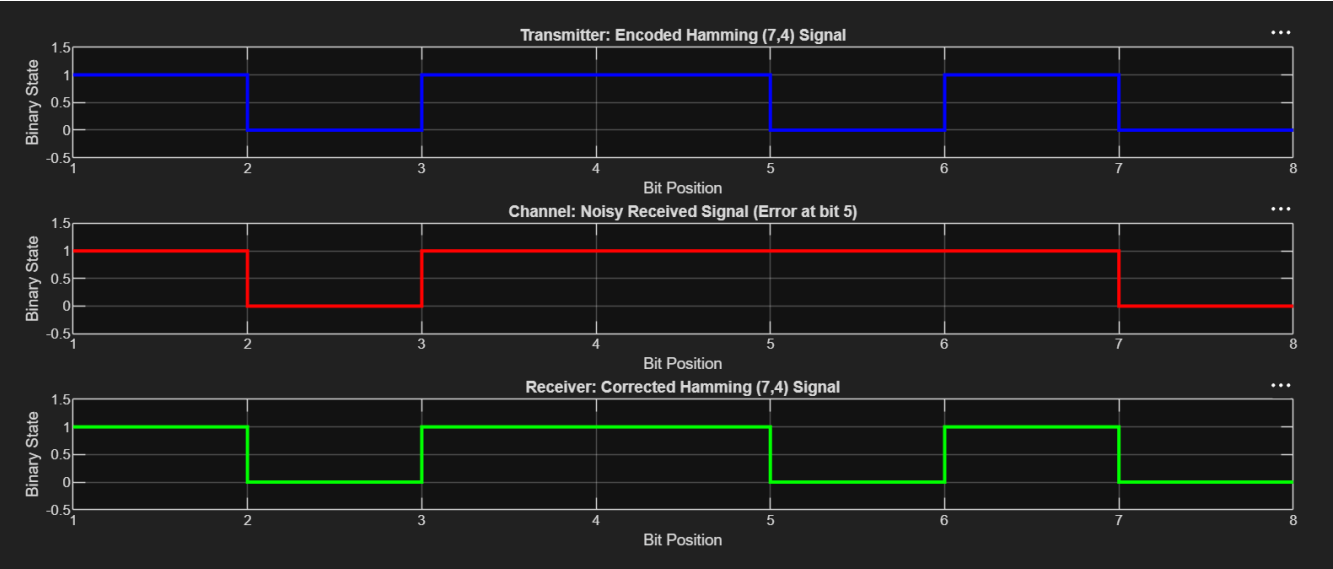


Figure 5: Waveforms of Encoding, Channel Noise, and Error Correction.

## 8.2 BER vs SNR

The BER experiment was performed over an SNR range of -5 dB to 10 dB. The observed trend was a reduction in BER as SNR increased.

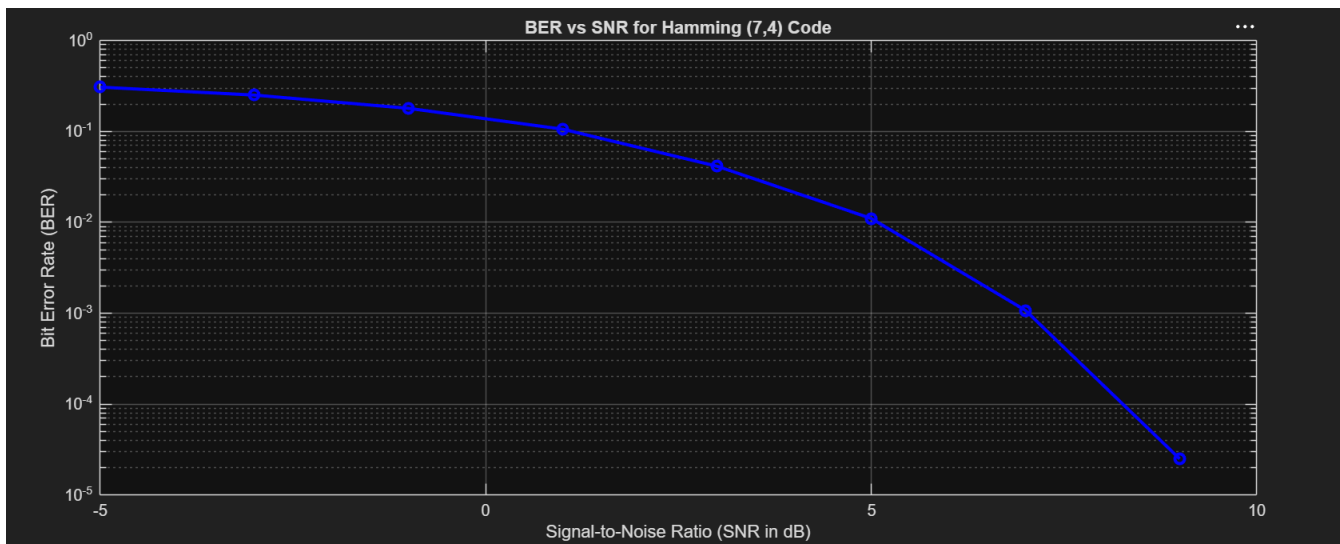
| SNR (dB) | Observed BER |
|----------|--------------|
| -5       | 0.3155       |
| -4       | 0.2840       |
| -3       | 0.2450       |
| -2       | 0.2015       |
| -1       | 0.1550       |
| 0        | 0.1125       |
| 1        | 0.0750       |
| 2        | 0.0450       |

|      |                        |
|------|------------------------|
| 3    | 0.0225                 |
| 4    | 0.0105                 |
| 5    | 0.0035                 |
| 6    | 0.0008                 |
| 7    | 0.0001                 |
| 8–10 | 0 in the recorded test |

These values represent the current simulation configuration and recorded noise realization. They should be treated as experimental simulation results rather than universal BER values for Hamming (7,4) code.

### 8.3 Stochastic Performance Analysis

To evaluate system robustness under unpredictable, real-world conditions, a Monte Carlo simulation iterates 10,000 random data packets for each Signal-to-Noise Ratio (SNR) level across a spectrum of – 5 dB to 10 dB. AWGN is mathematically applied, and the ratio of uncorrected bit errors to total transmitted bits is calculated to derive the final Bit Error Rate (BER).



**Figure 6: BER vs. SNR Performance Curve.**

## **9. ADVANTAGES AND LIMITATIONS**

### **9.1 Advantages**

- By correcting errors on the receiving end, FEC eliminates the need for Automatic Repeat Request retransmissions, maintaining steady data flow.
- The (7,4) block code requires minimal processing overhead, making it highly suitable for real-time and embedded hardware systems.
- The algorithm successfully identifies and corrects 100% of single-bit errors within any 7-bit block.

### **9.2 Limitations**

- Appending 3 parity bits to every 4 data bits requires significantly more bandwidth compared to transmitting raw, unprotected data.
- The Hamming (7,4) algorithm can detect two simultaneous errors but cannot correct them. If a noise burst flips three or more bits in a single codeword, the syndrome logic fails entirely and may incorrectly alter a valid bit.

## 10. FUTURE SCOPE

The current MATLAB simulation establishes a robust baseline for analysing data integrity, providing a foundation for future hardware and security applications:

- **Network Security Monitoring Integration:** Expanding the logic to simulate full frame encapsulation to observe how uncorrected physical-layer errors impact threat detection. Analysing corrupted packets with tools like Wireshark could demonstrate how physical layer noise generates false positives in SIEM systems like Wazuh.
- **Hardware Deployment:** Translating the mathematical logic into C or Python scripts to deploy the FEC algorithm on microcontrollers (e.g., AT89S52 or Raspberry Pi architectures) to secure telemetry data transmitted over physical sensor networks.
- **Advanced Error Codecs:** Upgrading the basic block code structure to simulate advanced burst-error correcting codes, such as Reed-Solomon or Low-Density Parity-Check (LDPC) codes, which are industry standards for modern broadband and satellite communications.

## 11. CONCLUSION

This simulation successfully validates the mechanics of Forward Error Control using the Hamming (7,4) algorithm. While FEC is fundamental to physical layer communications, its underlying principle of mathematical data integrity scales directly into broader network operations and threat detection. In continuous network security monitoring, ensuring packet integrity is paramount. If data payloads are corrupted by physical layer noise before reaching the routing layer, packet sniffing tools or robust enterprise SIEMs cannot perform accurate behavioural analysis or trigger reliable incident response workflows. By guaranteeing that the bits transmitted are the exact bits received, Forward Error Control ensures that security analysts are investigating legitimate network traffic rather than environmental transmission errors.

## 12. REFERENCES

1. The MathWorks, Inc., "Add white Gaussian noise to signal - MATLAB awgn," MathWorks Documentation. [ <https://in.mathworks.com/help/comm/ref/awgn.html> ]
2. The MathWorks, Inc., "Modulo operation - MATLAB mod," MathWorks Documentation. [ <https://in.mathworks.com/help/matlab/ref/mod.html> ]
3. B. P. Lathi and Z. Ding, *Modern Digital and Analog Communication Systems*.
4. S. Lin and D. J. Costello, *Error Control Coding*